

BICO DO DIA

09 — DIAGRAMA DE CLASSES UML

Produto: Bico do Dia

Documento: 09 — Diagrama de Classes UML

Versão: 1.0

Data: Agosto/2026

Status: Em elaboração

1. INTRODUÇÃO

O presente documento apresenta o **Diagrama de Classes UML do sistema Bico do Dia**.

O modelo foi elaborado a partir dos artefatos anteriores da engenharia de software:

- 01 — Visão do Produto;
- 02 — Levantamento de Requisitos;
- 03 — Documento de Requisitos de Software;
- 04 — Catálogo de Requisitos Funcionais;
- 05 — Requisitos Não Funcionais e Regras de Negócio;
- 06 — Histórias de Usuário;
- 07 — Catálogo de Casos de Uso;
- 08 — Diagramas de Casos de Uso.

O objetivo é identificar as principais classes do domínio, seus atributos, responsabilidades e relacionamentos.

2. OBJETIVO

O Diagrama de Classes tem como objetivos:

- representar as principais entidades do domínio;
 - identificar atributos;
 - identificar responsabilidades;
 - representar associações;
 - representar multiplicidades;
 - representar herança quando aplicável;
 - representar composição e agregação quando necessárias;
 - servir como base para a implementação orientada a objetos;
 - servir como referência para o MER;
 - manter rastreabilidade com os requisitos e casos de uso.
-

3. PRINCÍPIO DE MODELAGEM

O Bico do Dia será desenvolvido utilizando:

- Backend: Node.js;
- Frontend: JavaScript + React;
- Banco de dados: PostgreSQL através do Neon.

Entretanto, o Diagrama de Classes **não será construído diretamente como um espelho das tabelas do banco de dados.**

A ordem correta será:

Requisitos

↓

Casos de Uso

↓

Classes de Domínio

↓

Modelo Entidade-Relacionamento

↓

Modelo Relacional

↓

Banco PostgreSQL / Neon

Essa separação permite manter uma arquitetura mais organizada e facilita futuras evoluções da aplicação.

4. VISÃO GERAL DAS CLASSES

As principais classes identificadas são:

Usuários

- Usuario
- Cliente
- Prestador

- Administrador
- Endereco

Catálogo de serviços

- Categoria
- Especialidade
- Servico
- PortifolioItem
- Disponibilidade

Marketplace

- Solicitacao
- Proposta
- Contratacao
- Agendamento

Relacionamento

- Avaliacao
- Favorito
- Notificacao
- Denuncia

Administração

- Auditoria

5. CLASSE USUARIO

Responsabilidade

Representar a conta básica de uma pessoa cadastrada na plataforma.

```
Usuario
-----
- id: UUID
- nome: String
- email: String
- telefone: String
- senhaHash: String
- status: StatusUsuario
- dataCadastro: DateTime
- dataAtualizacao: DateTime
-----
+ autenticar()
+ atualizarDados()
```

```
+ alterarStatus()
+ verificarPermissao()
```

Responsabilidade

Representar o usuário que procura e contrata serviços.

```

Cliente
-----
- preferencias: String
-----
+ pesquisarServicos()
+ criarSolicitacao()
+ enviarProposta()
+ aceitarProposta()
+ cancelarSolicitacao()
+ avaliarPrestador()
+ adicionarFavorito()
+ removerFavorito()

```

Herança



Responsabilidade

```
Prestador
-----
- descricaoProfissional: String
```

```

- experiencia: String
- raioAtendimento: Decimal
- avaliacaoMedia: Decimal
- totalAvaliacoes: Integer
- perfilVerificado: Boolean
-----
+ cadastrarServico()
+ atualizarServico()
+ cadastrarEspecialidade()
+ configurarDisponibilidade()
+ consultarOportunidades()
+ enviarProposta()
+ atualizarContratacao()
+ informarConclusao()

```

O `Prestador` também especializa `Usuario`.

Herança

```

    Usuario
    ▲
    |
    Prestador

```

8. CLASSE ADMINISTRADOR

Responsabilidade

Representar o usuário responsável pela administração da plataforma.

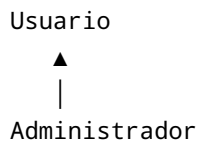
```

Administrador
-----
+ gerenciarUsuarios()
+ gerenciarCategorias()
+ gerenciarEspecialidades()
+ moderarConteudo()
+ moderarAvaliacoes()
+ consultarDashboard()
+ gerarRelatorios()
+ consultarAuditoria()
+ gerenciarDenuncias()

```

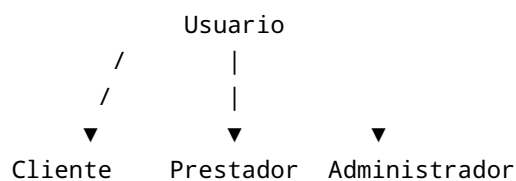
Também especializa `Usuario`.

Herança



9. GENERALIZAÇÃO DE USUÁRIOS

A estrutura principal será:



Essa abordagem evita duplicação dos atributos comuns.

10. CLASSE ENDERECO

Responsabilidade

Representar os endereços utilizados pelos usuários e solicitações.

```
Endereco
-----
- id: UUID
- cep: String
- logradouro: String
- numero: String
- complemento: String
- bairro: String
- cidade: String
- estado: String
- latitude: Decimal
- longitude: Decimal
- principal: Boolean
-----
+ validar()
+ definirComoPrincipal()
```

Relacionamentos

Um `Cliente` pode possuir vários endereços.

Cliente 1 — 0..* Endereco

Um endereço pode ser utilizado como referência para uma solicitação.

11. CLASSE CATEGORIA

Responsabilidade

Organizar os serviços oferecidos na plataforma.

```
Categoria
-----
- id: UUID
- nome: String
- descricao: String
- status: StatusCadastro
- dataCriacao: DateTime
-----
+ cadastrar()
+ atualizar()
+ ativar()
+ inativar()
```

Uma categoria pode possuir diversos serviços.

Categoria 1 — 0..* Servico

12. CLASSE ESPECIALIDADE

Responsabilidade

Representar uma especialidade profissional.

```
Especialidade
-----
- id: UUID
- nome: String
```

```

- descricao: String
- status: StatusCadastro
-----
+ cadastrar()
+ atualizar()
+ ativar()
+ inativar()

```

Um prestador pode possuir várias especialidades.

Uma especialidade pode pertencer a vários prestadores.

```

Prestador 0..* ——— 0..* Especialidade

```

Esse relacionamento será posteriormente transformado em uma entidade associativa no modelo relacional.

13. CLASSE SERVICO

Responsabilidade

Representar um serviço oferecido por um prestador.

```

Servico
-----
- id: UUID
- nome: String
- descricao: String
- precoInicial: Decimal
- precoFinal: Decimal
- duracaoEstimada: Integer
- status: StatusServico
- dataCadastro: DateTime
- dataAtualizacao: DateTime
-----
+ cadastrar()
+ atualizar()
+ ativar()
+ inativar()

```

Relacionamentos

Um prestador oferece vários serviços.

Prestador 1 — 0..* Servico

Cada serviço pertence a uma categoria.

Categoria 1 — 0..* Servico

14. CLASSE PORTFOLIOITEM

Responsabilidade

Representar um item do portfólio profissional do prestador.

```
PortifolioItem
-----
- id: UUID
- titulo: String
- descricao: String
- urlArquivo: String
- tipoArquivo: String
- dataCadastro: DateTime
-----
+ adicionar()
+ atualizar()
+ remover()
```

Relacionamento

Prestador 1 — 0..* PortifolioItem

15. CLASSE DISPONIBILIDADE

Responsabilidade

Representar os horários em que o prestador está disponível.

```
Disponibilidade
-----
- id: UUID
- diaSemana: Integer
- horaInicio: Time
```

```

- horaFim: Time
- ativo: Boolean
-----
+ configurar()
+ atualizar()
+ ativar()
+ desativar()
+ verificarDisponibilidade()

```

Relacionamento

Prestador 1 — 0..* Disponibilidade

16. CLASSE SOLICITACAO

Responsabilidade

Representar uma necessidade de serviço publicada por um cliente.

```

Solicitacao
-----
- id: UUID
- titulo: String
- descricao: String
- dataDesejada: Date
- horaDesejada: Time
- status: StatusSolicitacao
- dataCriacao: DateTime
- dataAtualizacao: DateTime
-----
+ criar()
+ atualizar()
+ publicar()
+ cancelar()
+ encerrar()

```

Relacionamentos

Cliente 1 — 0..* Solicitacao

Uma solicitação pode receber várias propostas.

Solicitacao 1 — 0..* Proposta

17. CLASSE PROPOSTA

Responsabilidade

Representar a proposta enviada por um prestador para uma solicitação.

```
Proposta
-----
- id: UUID
- valor: Decimal
- descricao: String
- prazoExecucao: Integer
- validade: DateTime
- status: StatusProposta
- dataCriacao: DateTime
- dataAtualizacao: DateTime
-----
+ criar()
+ atualizar()
+ aceitar()
+ recusar()
+ cancelar()
+ expirar()
```

Relacionamentos

Prestador 1 — 0..* Proposta
Solicitacao 1 — 0..* Proposta

Uma proposta pertence a um único prestador e a uma única solicitação.

18. CLASSE CONTRATAÇÃO

Responsabilidade

Representar o vínculo formal entre cliente e prestador após a aceitação de uma proposta.

Contratacao

```
-----  
- id: UUID  
- valorContratado: Decimal  
- dataContratacao: DateTime  
- dataInicio: DateTime  
- dataConclusao: DateTime  
- status: StatusContratacao  
- observacoes: String  
-----
```

```
+ criar()  
+ iniciar()  
+ atualizar()  
+ concluir()  
+ cancelar()
```

Relacionamentos

Cliente 1 — 0..* Contratacao

Prestador 1 — 0..* Contratacao

Proposta 1 — 0..1 Contratacao

Uma proposta aceita pode gerar uma contratação.

19. CLASSE AGENDAMENTO

Responsabilidade

Representar a data e horário definidos para execução de um serviço.

Agendamento

```
-----  
- id: UUID  
- data: Date  
- horaInicio: Time  
- horaFim: Time  
- status: StatusAgendamento  
- observacoes: String  
-----
```

```
+ agendar()  
+ reagendar()  
+ confirmar()
```

```
+ cancelar()  
+ concluir()
```

Relacionamento

Contratacao 1 — 0..1 Agendamento

20. CLASSE AVALIACAO

Responsabilidade

Representar a avaliação realizada após a conclusão do serviço.

```
Avaliacao  
-----  
- id: UUID  
- nota: Integer  
- comentario: String  
- status: StatusAvaliacao  
- dataCriacao: DateTime  
- dataAtualizacao: DateTime  
-----  
+ avaliar()  
+ editar()  
+ remover()  
+ validar()
```

Relacionamentos

Cliente 1 — 0..* Avaliacao
Prestador 1 — 0..* Avaliacao
Contratacao 1 — 0..1 Avaliacao

Uma avaliação deve estar vinculada a uma contratação concluída.

21. CLASSE FAVORITO

Responsabilidade

Representar o relacionamento entre um cliente e um prestador favorito.

Favorito

- id: UUID
- dataCriacao: DateTime

+ adicionar()
+ remover()

Relacionamento

Cliente 1 — 0..* Favorito

Prestador 1 — 0..* Favorito

22. CLASSE NOTIFICACAO

Responsabilidade

Representar mensagens e alertas enviados aos usuários.

Notificacao

- id: UUID
- titulo: String
- mensagem: String
- tipo: TipoNotificacao
- lida: Boolean
- dataCriacao: DateTime
- dataLeitura: DateTime

+ enviar()
+ marcarComoLida()
+ marcarComoNaoLida()

Relacionamento

Usuario 1 — 0..* Notificacao

23. CLASSE DENUNCIA

Responsabilidade

Representar uma denúncia realizada por um usuário.

```
Denuncia
-----
- id: UUID
- motivo: String
- descricao: String
- status: StatusDenuncia
- dataCriacao: DateTime
- dataTratamento: DateTime
-----
+ registrar()
+ analisar()
+ aprovar()
+ rejeitar()
+ encerrar()
```

Uma denúncia deve estar associada ao usuário que a realizou.

```
Usuario 1 ——— 0..* Denuncia
```

A denúncia poderá estar relacionada a conteúdos diferentes, como avaliações ou outros conteúdos moderáveis.

24. CLASSE AUDITORIA

Responsabilidade

Registrar operações relevantes realizadas na plataforma.

```
Auditoria
-----
- id: UUID
- acao: String
- entidade: String
- entidadeId: UUID
- dadosAnteriores: JSON
- dadosNovos: JSON
- ip: String
- dataHora: DateTime
```

+ registrar()
+ consultar()

Relacionamento

Usuario 1 ——— 0..* Auditoria

25. ENUMERAÇÕES

Para evitar utilização indiscriminada de strings no domínio, serão utilizadas enumerações.

StatusUsuario

ATIVO
INATIVO
BLOQUEADO
PENDENTE

StatusCadastro

ATIVO
INATIVO

StatusServico

ATIVO
INATIVO

StatusSolicitacao

RASCUNHO
PUBLICADA
EM_NEGOCIACAO
CONTRATADA
CANCELADA
ENCERRADA

StatusProposta

ENVIADA
ACEITA
RECUSADA
CANCELADA
EXPIRADA

StatusContratacao

PENDENTE
CONFIRMADA
EM_ANDAMENTO
CONCLUIDA
CANCELADA

StatusAgendamento

PENDENTE
CONFIRMADO
REALIZADO
CANCELADO
REAGENDADO

StatusAvaliacao

PUBLICADA
OCULTA
REMOVIDA

StatusDenuncia

ABERTA
EM_ANALISE
PROCEDENTE
IMPROCEDENTE
ENCERRADA

TipoNotificacao

NOVA_SOLICITACAO
NOVA_PROPOSTA

PROPOSTA_ACEITA
PROPOSTA_RECUSADA
CONTRATAÇÃO
AGENDAMENTO
CANCELAMENTO
CONCLUSÃO
AVALIAÇÃO
SISTEMA

26. RELACIONAMENTOS PRINCIPAIS

A estrutura central do domínio pode ser representada da seguinte maneira:

Usuario

- Cliente
- Prestador
- Administrador

Cliente

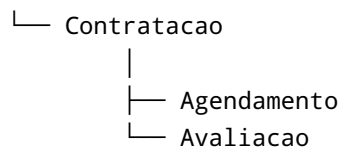
- Endereco
- Solicitacao
- Favorito
- Avaliacao
- Contratacao

Prestador

- Especialidade
- Servico
- PortifolioItem
- Disponibilidade
- Proposta
- Contratacao
- Avaliacao
- Favorito

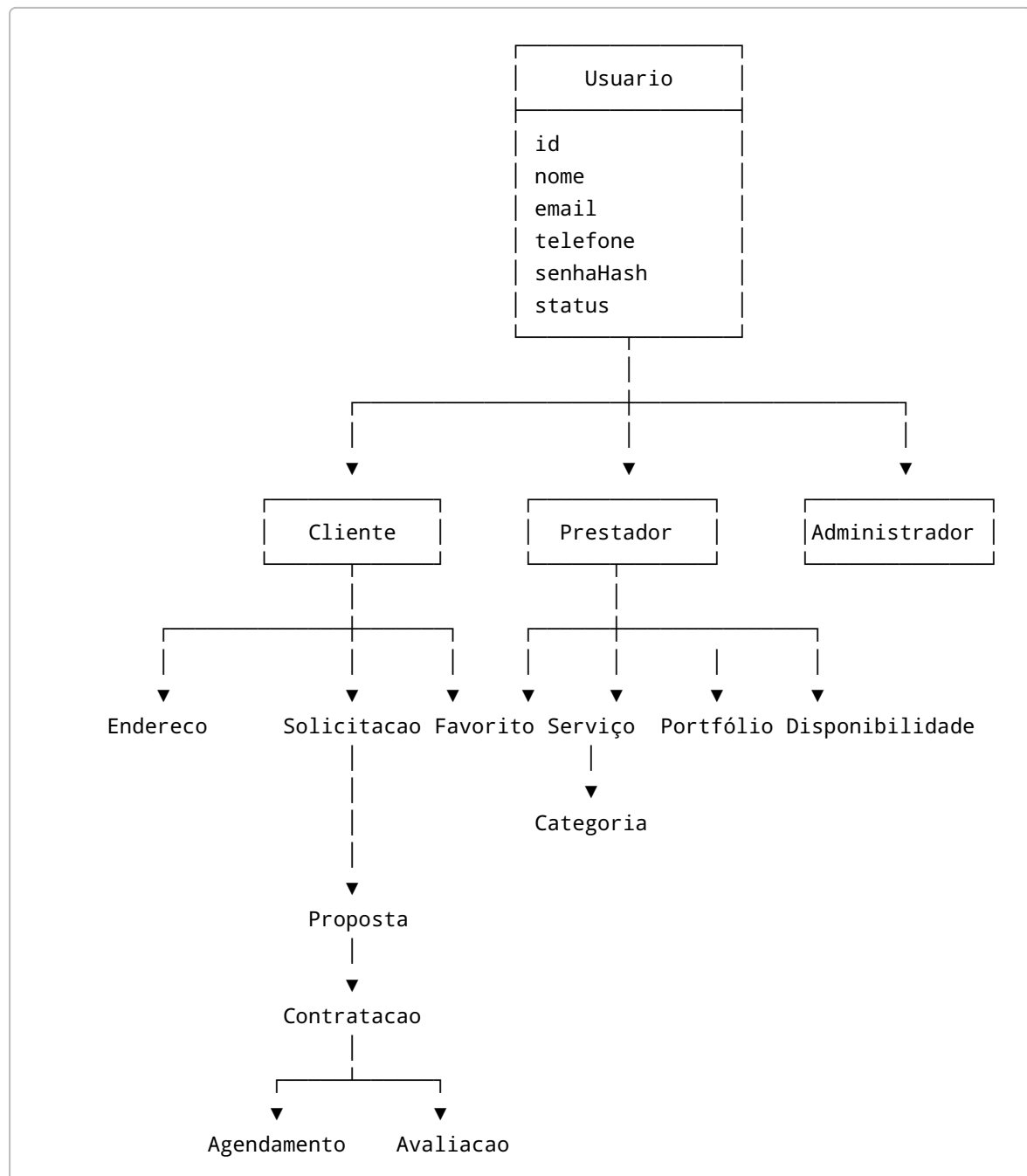
Solicitacao

- Proposta



27. DIAGRAMA DE CLASSES — VISÃO GERAL

A representação conceitual consolidada é:



28. MODELO DE RELACIONAMENTOS E MULTIPLICIDADES

Classe A	Multiplicidade	Classe B	Relação
Usuario	1	Cliente	Herança
Usuario	1	Prestador	Herança
Usuario	1	Administrador	Herança
Cliente	1	Endereco	0..*
Cliente	1	Solicitacao	0..*
Prestador	1	Servico	0..*
Prestador	1	PortifolioItem	0..*
Prestador	1	Disponibilidade	0..*
Prestador	0..*	Especialidade	0..*
Categoria	1	Servico	0..*
Solicitacao	1	Proposta	0..*
Prestador	1	Proposta	0..*
Proposta	1	Contratacao	0..1
Cliente	1	Contratacao	0..*
Prestador	1	Contratacao	0..*
Contratacao	1	Agendamento	0..1
Cliente	1	Avaliacao	0..*
Prestador	1	Avaliacao	0..*
Contratacao	1	Avaliacao	0..1
Cliente	1	Favorito	0..*
Prestador	1	Favorito	0..*
Usuario	1	Notificacao	0..*
Usuario	1	Denuncia	0..*
Usuario	1	Auditoria	0..*

29. RELACIONAMENTOS N:N

Existem dois relacionamentos que exigem atenção especial.

29.1 Prestador × Especialidade

Um prestador pode possuir várias especialidades.

Uma especialidade pode estar associada a vários prestadores.

Portanto:

Prestador N — N Especialidade

No banco de dados, deverá existir uma entidade associativa:

PrestadorEspecialidade

29.2 Cliente × Prestador através de Favorito

Embora conceitualmente o relacionamento seja:

Cliente N — N Prestador

a classe `Favorito` representa o relacionamento e permite armazenar informações adicionais, como:

- data de inclusão;
- data de remoção, se necessário;
- estado do favorito.

Portanto:

Cliente 1 — N Favorito N — 1 Prestador

30. COMPOSIÇÃO E AGREGAÇÃO

Alguns relacionamentos podem ser interpretados como composição de domínio.

Prestador → PortfolioItem

Um item de portfólio pertence ao perfil de um prestador.

Prestador ◆ — PortfolioItem

Se o portfólio do prestador for removido definitivamente, seus itens também deixam de existir no contexto da aplicação.

Contratacao → Agendamento

O agendamento existe em função da contratação.

Contratacao ♦ — Agendamento

31. REGRAS IMPORTANTES DO DOMÍNIO

O modelo deverá respeitar as seguintes regras:

Regra 1

Um usuário deve possuir uma conta válida para acessar recursos protegidos.

Regra 2

Somente prestadores podem cadastrar serviços profissionais.

Regra 3

Um serviço deve possuir uma categoria válida.

Regra 4

Uma solicitação pertence a um cliente.

Regra 5

Uma solicitação pode receber várias propostas.

Regra 6

Uma proposta pertence a um único prestador.

Regra 7

Uma proposta aceita pode originar uma contratação.

Regra 8

Uma contratação deve possuir um cliente e um prestador.

Regra 9

Um agendamento somente pode ser criado para uma contratação válida.

Regra 10

Uma avaliação somente poderá ocorrer após a conclusão da contratação.

Regra 11

Um cliente não deve avaliar uma contratação que não realizou.

Regra 12

A reputação do prestador deve considerar somente avaliações válidas.

Regra 13

Usuários bloqueados não podem executar operações restritas.

Regra 14

Alterações administrativas relevantes devem ser registradas em auditoria.

32. RASTREABILIDADE COM OS CASOS DE USO

Caso de Uso	Principais Classes
UC001 Criar Conta	Usuario, Cliente, Prestador
UC002 Autenticar Usuário	Usuario
UC003 Gerenciar Conta	Usuario
UC004 Recuperar Credenciais	Usuario
UC005 Perfil Cliente	Cliente, Endereco
UC006 Perfil Prestador	Prestador, Especialidade
UC007 Categorias	Categoria
UC008 Especialidades	Especialidade
UC009 Serviços	Servico, Categoria
UC010 Portfólio	PortfolioItem
UC011 Disponibilidade	Disponibilidade
UC012 Pesquisa	Servico, Prestador, Categoria
UC013 Perfil Prestador	Prestador, Servico, Avaliacao

Caso de Uso	Principais Classes
UC014 Solicitação	Solicitacao, Endereco
UC015 Gerenciar Solicitação	Solicitacao
UC016 Oportunidades	Solicitacao, Prestador
UC017 Proposta	Proposta, Solicitacao, Prestador
UC018 Gerenciar Propostas	Proposta
UC019 Contratação	Contratacao, Proposta
UC020 Gerenciar Contratação	Contratacao
UC021 Agenda	Agendamento, Disponibilidade
UC022 Avaliação	Avaliacao, Contratacao
UC023 Reputação	Avaliacao, Prestador
UC024 Favoritos	Favorito, Cliente, Prestador
UC025 Notificações	Notificacao, Usuario
UC026 Usuários	Usuario
UC027 Conteúdo	PortfolioItem, Servico
UC028 Avaliações	Avaliacao
UC029 Categorias/Especialidades	Categoria, Especialidade
UC030 Dashboard	Usuario, Solicitacao, Proposta, Contratacao
UC031 Relatórios	classes de domínio
UC032 Auditoria	Auditoria, Usuario
UC033 Denúncias	Denuncia, Usuario
UC034 Status da Plataforma	configuração da plataforma

33. RASTREABILIDADE COM OS REQUISITOS

O Diagrama de Classes possui relação direta com os requisitos funcionais que envolvem:

- gerenciamento de usuários;
- gerenciamento de perfis;
- gerenciamento de serviços;
- gerenciamento de categorias;
- gerenciamento de especialidades;
- gerenciamento de solicitações;
- gerenciamento de propostas;
- contratação;

- agenda;
- avaliações;
- reputação;
- favoritos;
- notificações;
- denúncias;
- administração;
- auditoria.

Os requisitos não funcionais relacionados à segurança, desempenho, disponibilidade e escalabilidade deverão ser tratados principalmente na arquitetura da solução, e não incorporados indevidamente como atributos de entidades de negócio.

34. DECISÕES DE MODELAGEM

34.1 Usuário como classe base

A classe `Usuario` concentra os dados comuns.

Isso evita duplicação entre:

- Cliente;
- Prestador;
- Administrador.

34.2 Cliente e Prestador como especializações

O sistema possui comportamentos diferentes para quem contrata e para quem presta serviços.

Por isso, a especialização é adequada ao domínio.

34.3 Contratação como entidade própria

Não devemos relacionar diretamente Cliente e Prestador como se a contratação fosse apenas uma associação.

A contratação possui informações próprias:

- valor;
- datas;
- status;
- observações;
- vínculo com proposta;
- agendamento;
- avaliação.

Portanto, deve ser uma classe própria.

34.4 Proposta como entidade própria

A proposta representa uma negociação entre prestador e cliente.

Ela possui:

- valor;
- descrição;
- prazo;
- validade;
- status.

Portanto, também deve ser uma classe independente.

34.5 Avaliação vinculada à contratação

A avaliação não deve existir de maneira independente.

Ela deve estar relacionada a uma contratação concluída.

Isso permite garantir a regra:

Somente serviços efetivamente realizados podem gerar avaliações válidas.

35. PREPARAÇÃO PARA O MER

O Diagrama de Classes já permite identificar várias futuras entidades do banco de dados.

A primeira projeção para o MER será:

```
USUARIO
CLIENTE
PRESTADOR
ADMINISTRADOR
ENDERECO

CATEGORIA
ESPECIALIDADE
PRESTADOR_ESPECIALIDADE
SERVICO
PORTFOLIO_ITEM
DISPONIBILIDADE
```

SOLICITACAO
PROPOSTA
CONTRATACAO
AGENDAMENTO

AVALIACAO
FAVORITO
NOTIFICACAO
DENUNCIA
AUDITORIA

Essa lista **não deve ser considerada ainda como o modelo definitivo do banco.**

O MER será construído após refinarmos:

- cardinalidades;
- chaves;
- atributos;
- dependências;
- entidades associativas;
- integridade referencial;
- normalização.

36. PREPARAÇÃO PARA A IMPLEMENTAÇÃO NODE.JS

O modelo também servirá como base para a implementação do backend.

Uma possível organização futura será:

```
src/  
|  
|— modules/  
|  
|   |— usuario/  
|   |— cliente/  
|   |— prestador/  
|   |— categoria/  
|   |— especialidade/  
|   |— servico/  
|   |— solicitacao/  
|   |— proposta/  
|   |— contratacao/  
|   |— agendamento/  
|   |— avaliacao/  
|   |— favorito/
```

```
|
|   ├── notificacao/
|   ├── denuncia/
|   └── auditoria/
|
|── controllers/
|── services/
|── repositories/
|── middlewares/
|── routes/
└── config/
```

Essa estrutura será refinada posteriormente durante o documento de arquitetura.

37. CONSIDERAÇÕES SOBRE ESCALABILIDADE

Como definido na Visão do Produto, o Bico do Dia deve ser concebido desde o início considerando crescimento futuro.

O modelo de domínio permite evolução para:

- aplicação mobile;
- múltiplas regiões;
- geolocalização;
- notificações push;
- pagamentos;
- planos para prestadores;
- serviços premium;
- verificação profissional;
- chat entre cliente e prestador;
- histórico de preços;
- cupons;
- promoções;
- recomendações;
- inteligência de matching entre cliente e prestador.

Essas funcionalidades **não serão incorporadas ao MVP sem necessidade**, mas a modelagem atual não deve impedir sua futura inclusão.

38. CONSIDERAÇÕES FINAIS

O Diagrama de Classes estabelece o núcleo estrutural do domínio do Bico do Dia.

O modelo atual possui como núcleo:

Usuario

↓

Cliente / Prestador / Administrador

e, no fluxo comercial:

Solicitacao

↓

Proposta

↓

Contratacao

↓

Agendamento

↓

Avaliacao

Ao redor desse núcleo estão:

- serviços;
- categorias;
- especialidades;
- portfólio;
- disponibilidade;
- favoritos;
- notificações;
- denúncias;
- auditoria.

Essa estrutura fornece uma base sólida para a próxima etapa de modelagem.

39. PRÓXIMO ARTEFATO

O próximo documento recomendado será:

10 — DIAGRAMAS DE SEQUÊNCIA UML

A partir dos casos de uso e das classes agora identificadas, deverão ser modelados os principais fluxos de interação.

Recomenda-se inicialmente criar os seguintes diagramas:

DS01 — Cadastro e Autenticação

```
graph TD;
  U[Usuário] --> FR[Frontend React];
  FR --> API[API Node.js];
  API --> SA[Serviço de Autenticação];
  SA --> B[Banco Neon];
```

DS02 — Pesquisa de Serviço

```
graph TD;
  C[Cliente] --> F[Frontend];
  F --> API[API];
  API --> SP[Serviço de Pesquisa];
  SP --> B[Banco];
  B --> PS[Prestadores/Serviços];
```

DS03 — Criação de Solicitação

```
graph TD;
  C[Cliente] --> F[Frontend];
  F --> API[API];
  API --> S[Solicitação];
  S --> B[Banco];
  B --> N[Notificação];
```

DS04 — Envio de Proposta

```
graph TD;
  P[Prestador] --> ;
```

```
graph TD; Frontend --> API; API --> Proposta; Proposta --> Solicitação; Solicitação --> Banco; Banco --> Notificação do Cliente;
```

DS05 — Contratação

```
graph TD; Cliente --> Proposta; Proposta --> Contratação; Contratação --> Agenda; Agenda --> Notificação;
```

DS06 — Conclusão e Avaliação

```
graph TD; Prestador --> Conclusão; Conclusão --> Contratação; Contratação --> Cliente; Cliente --> Avaliação; Avaliação --> Reputação;
```

Depois dos Diagramas de Sequência, poderemos avançar para os **Diagramas de Atividade** e, em seguida, transformar o modelo de classes em um **MER completo**, preparando efetivamente a estrutura do PostgreSQL no Neon.